

Обучающие материалы / [UNIX/Linux](#) / Как создать раздел в Linux (fdisk и parted)

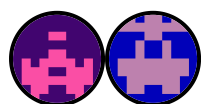
Учебник

Как создать раздел в Linux (fdisk и parted)

Обновлено 29 июня 2026 года



UNIX/Linux



Автор: [e09330e1749e4191a9dfe441e5129f](#) и Маникандан Куруп



Products



Solutions



Developers



Partners



Pricing

Log in 

Sign up 

Обучающие материалы

Вопросы

Документация по продукту

 Поиск

Создание раздела в [Linux](#) позволяет разделить один физический диск на отдельные независимые секции, которые система воспринимает как разные диски. С помощью разбиения на разделы можно выделить специальное пространство для определенных целей, изолировать операционную систему от пользовательских данных, настроить [раздел подкачки](#), запустить на одном компьютере несколько операционных систем и ограничить ущерб в случае повреждения файловой системы. Поскольку каждый раздел ведет себя как отдельный диск, проблема на одном из них не затронет данные на других.

Из этого руководства вы узнаете, как создать раздел в Linux из командной строки с помощью двух самых распространенных инструментов: `fdisk` и `parted`. Вы сможете выбрать между таблицей разделов MBR и GPT, безопасно определить нужный диск, создать новый раздел и задать его тип, отформатировать его с помощью файловой системы, например `ext4` или `xfs`, временно смонтировать, сделать монтирование постоянным с помощью `/etc/fstab`, удалить раздел и исправить наиболее часто возникающие ошибки.

Предупреждение: Разбиение на разделы и форматирование — это разрушительные операции. Запись новой таблицы разделов или выполнение `mkfs` на неподходящем устройстве приведет к безвозвратному удалению данных. Всегда проверяйте целевой диск с помощью [lsblk](#) и `sudo fdisk -l` перед внесением каких-либо изменений и сначала создавайте резервные копии важных данных.

Ключевые рекомендации на вынос

- Разбиение на разделы позволяет разделить один физический диск на независимые секции, каждая из которых может содержать собственную файловую систему и монтироваться отдельно.
- Для хранения разделов на диске необходима таблица разделов. Два наиболее распространенных типа — MBR (msdos) и GPT.
- MBR поддерживает диски объемом до 2,2 ТБ (2 199 023 255 040 байт с 512-байтовыми секторами) и максимум четыре основных раздела, поэтому она подходит для старых систем с BIOS и небольших дисков.
- GPT снимает ограничение в 2 ТБ, по умолчанию поддерживает до 128 разделов и является оптимальным вариантом для больших дисков и систем UEFI.
- [fdisk](#) — это интерактивный инструмент, доступный по умолчанию в большинстве дистрибутивов и поддерживающий как MBR, так и GPT. Поддержка GPT была представлена как экспериментальная функция в `util-linux 2.23` и стала стабильной в более поздних выпусках; современные дистрибутивы поставляются с полноценной версией.
- [parted](#) изначально поддерживает большие диски и может работать в неинтерактивном режиме, что делает его лучшим выбором для написания скриптов и автоматизации.
- Создание раздела — это только первый шаг: необходимо отформатировать его с помощью файловой системы (например, `mkfs.ext4` или `mkfs.xfs`) и смонтировать, прежде чем вы сможете хранить на нем данные.
- Добавление записи на основе UUID в `/etc/fstab` позволяет автоматически монтировать раздел при каждой загрузке системы.

Прежде чем создавать раздел, убедитесь, что у вас есть следующее.

Для работы вам понадобится система Linux со вторым диском или нераспределенным пространством, а также доступ к учетной записи пользователя с `sudo` привилегиями, чтобы вы могли выполнять административные команды. Если вам нужно сначала все настроить, следуйте руководству по [созданию нового пользователя с правами sudo](#). Вам также должно быть удобно работать в [терминале Linux](#), поскольку на каждом этапе этого руководства используется командная строка.

И `fdisk` и `parted` входят в состав стандартных утилит большинства дистрибутивов. `fdisk` поставляется с пакетом `util-linux` и почти всегда присутствует в системе, в то время как `parted` может потребоваться установить в минимальной системе с `sudo apt install parted` в [Ubuntu](#) и [Debian](#) или `sudo dnf install parted` в Fedora, [Rocky Linux](#) и других дистрибутивах на основе Red Hat.

Understanding partition tables: GPT vs MBR ^

A disk cannot hold partitions until it has a partition table, the small structure at the start of the disk that records where each partition begins and ends. The two formats you will encounter are MBR (also shown as `msdos`) and GPT, and choosing the right one matters before you create anything.

MBR (Master Boot Record) is the older standard. It can address disks only up to roughly 2.2 TB (2,199,023,255,040 bytes with 512-byte sectors) and allows a maximum of four primary partitions. To exceed four, MBR lets you turn one primary partition into an extended partition that holds additional logical partitions.

GPT (GUID Partition Table) is the modern replacement defined by the UEFI standard. It removes the 2 TB limit (addressing disks up to around 9.4 zettabytes), supports up to 128 partitions by default without extended or logical partitions, and keeps a backup copy of the table at the end of the disk for resilience.

As a quick rule, choose **MBR** for legacy BIOS systems or disks under 2 TB that need compatibility with older systems, and choose **GPT** for any disk larger than 2 TB, for UEFI systems, or simply as a modern default.

You can check which partition table a disk already uses before making changes. The following command prints disk details, including the partition table type:

```
$ sudo fdisk -l /dev/vda
```

Look for the `Disklabel type` line in the output, which reports either `dos` (MBR) or `gpt` :

Output

Copy

```
Disk /dev/vda: 160 GiB, 171798691840 bytes, 335544320 sectors
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disklabel type: gpt
Disk identifier: DCDD7CFE-DCA3-4181-877E-B1DFCF70E9AA
```

Choosing the right partitioning tool ^

Linux gives you several tools for the same job, and the best one depends on your disk size, whether you want to script the process, and whether you have a desktop environment. The three most common options are `fdisk` , an interactive command-line tool installed by default on nearly every distribution that handles both MBR and GPT (GPT support was introduced as experimental in util-linux 2.23 and is stable in all modern releases); `parted` , a command-line tool that supports large disks natively and can run non-interactively with the `--script` flag, which makes it well suited for automation; and `GParted` , a graphical front end that is convenient on desktops with a GUI but is not typically available on headless servers.

The following table compares the three tools so you can pick the right one for your situation:

Feature	fdisk	parted	GParted
Interface	Interactive CLI	CLI (interactive or scripted)	Graphical (GUI)

Feature	fdisk	parted	GParted
Partition tables	MBR and GPT	MBR and GPT	MBR and GPT
Disks larger than 2 TB	Yes (with GPT)	Yes	Yes
Scripting and automation	Limited	Yes (<code>--script</code>)	No
Default availability	Installed on most distros	Often preinstalled, may need install	Desktop only
Best for	Quick interactive edits on a server	Automation, scripting, and large disks	Visual editing on a desktop

How to identify your target disk using `lsblk` and `fdisk` [^] -1

Before you partition anything, you must identify the correct disk, because choosing the wrong device can destroy data. Two commands give you a clear picture of the disks and partitions on your system.

The `lsblk` command lists all block devices in an easy-to-read tree, showing each disk and its existing partitions along with their sizes and mount points:

Copy

```
$ lsblk
```

The output shows your disks and any partitions beneath them:

Output

Copy

NAME	MAJ:MIN	RM	SIZE	RO	TYPE	MOUNTPPOINTS
sda	8:0	0	160G	0	disk	
sda1	8:1	0	159G	0	part	/
sda14	8:14	0	4M	0	part	
sda15	8:15	0	106M	0	part	/boot/efi
sda16	8:16	0	913M	0	part	/boot
sdb	8:16	0	20G	0	disk	

In this example, `sdb` is a 20 GiB disk with no partitions, which makes it the disk to work on. For a more detailed view that includes partition tables, types, and sizes, use `fdisk -l`:

Copy

```
$ sudo fdisk -l
```

Output

Copy

```
Disk /dev/sda: 160 GiB, 171798691840 bytes, 335544320 sectors
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disklabel type: gpt
Disk identifier: DCDD7CFE-DCA3-4181-877E-B1DFCF70E9AA

Device      Start      End      Sectors  Size Type
/dev/sda1   2099200 335544286 333445087 159G Linux filesystem
/dev/sda14    2048    10239    8192    4M BIOS boot
/dev/sda15   10240   227327   217088  106M EFI System
/dev/sda16  227328  2097152  1869825  913M Linux extended boot

Partition table entries are not in disk order.

Disk /dev/sdb: 20 GiB, 21474836480 bytes, 41943040 sectors
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
```

You will be prompted for your password to confirm your `sudo` privileges. The command lists every disk and its current partition layout, which lets you confirm the device path (for example `/dev/sdb`) before you continue.

How to create a partition using `fdisk`

`fdisk` is a text-based utility for viewing and managing disk partitions, and its interactive command mode is the classic way to create a partition in Linux. The steps below walk through a complete session, from opening the disk to writing the changes. Start by opening the target disk in command mode, replacing `/dev/sdb` with the disk you identified earlier:

Copy

```
$ sudo fdisk /dev/sdb
```

Once you are in command mode, `fdisk` waits for single-character commands. The interface can be confusing at first, so press `m` at any time to see the full list of available commands:

Copy

Output

```
Welcome to fdisk (util-linux 2.40.2).
Changes will remain in memory only, until you decide to write them.
Be careful before using the write command.
```

```
Command (m for help):
```

Output

Copy

Help:

GPT

```
M   enter protective/hybrid MBR
```

Generic

```
d   delete a partition
```



```
F  list free unpartitioned space
l  list known partition types
n  add a new partition
p  print the partition table
t  change a partition type
v  verify the partition table
i  print information about a partition

Misc
m  print this menu
x  extra functionality (experts only)

Script
I  load disk layout from sfdisk script file
O  dump disk layout to sfdisk script file

Save & Exit
w  write table to disk and exit
q  quit without saving changes

Create a new label
g  create a new empty GPT partition table
G  create a new empty SGI (IRIX) partition table
o  create a new empty MBR (DOS) partition table
s  create a new empty Sun partition table
```

Command (m for help):

If the disk has no partition table yet, create one before adding partitions. Press `g` to create a new empty GPT partition table, or `o` to create an MBR (`dos`) table, depending on which type you chose earlier.

To create the partition, type `n` and press `ENTER` . On an MBR disk, `fdisk` first asks for the partition type: choose `p` for a primary partition or `l` for a logical partition. For this tutorial, create a primary partition by entering `p` . On a GPT disk this prompt does not appear, because GPT does not use the primary, extended, and logical distinction.

Next, `fdisk` asks for the starting sector; press `ENTER` to accept the first free sector. It then asks for the last sector, where you can press `ENTER` to use all remaining space or specify a size such as `+10G` for a 10 GB partition or `+512M` for 512 MB. A number with no unit is interpreted as sectors, so `+1024` means 1024 sectors from the start.

On MBR disks, `fdisk` assigns the new partition the type `Linux` (hex code `83`) by default. On GPT disks, the equivalent default type is `Linux filesystem` , identified by a GUID rather than a hex code. To set a different type, such as `Linux LVM` , use the `t`

command and press `L` to list all available types and aliases. On a GPT disk, enter the text alias `lvm` to set the partition type to `Linux LVM` :

Output

Copy

```
Command (m for help): t
Partition number (1,14-16, default 16): 1
Partition type or alias (type L to list all): 44

Changed type of partition 'Linux filesystem' to 'Linux LVM'.
```

Before writing anything to disk, review your work. All changes so far live only in memory, so press `p` to print the current partition table and confirm it looks correct:

Output

Copy

```
Command (m for help): p
Disk /dev/sdb: 20 GiB, 21474836480 bytes, 41943040 sectors
Disklabel type: gpt
Device      Start      End  Sectors  Size Type
/dev/sdb1   2048 20973567 20971520  10G Linux LVM
```

When you are satisfied, write the changes to disk with `w` . This is the point of no return, so make sure the target is correct:

Output

Copy

```
Command (m for help): w
The partition table has been altered.
Calling ioctl() to re-read partition table.
Syncing disks.
```

If you decide not to keep your changes, press `q` instead of `w` to quit `fdisk` without writing anything, leaving the disk untouched.

How to create a partition using parted



`parted` is well suited for automation and scripting, and also works well for interactive use. You can run it interactively or pass commands directly on the command line. The interactive session below creates a GPT table and a single partition that fills the disk.

Open the target disk with `parted` :

Copy

```
$ sudo parted /dev/sdb
```

Inside the `parted` prompt, set the partition table type with `mklabel` . Use `gpt` for a GPT table or `msdos` for an MBR table:

Copy

```
$ (parted) mklabel gpt
```

Create the partition with `mkpart` . The following command creates a partition named `data` that spans the entire disk, from the start (`0%`) to the end (`100%`):

Copy

```
$ (parted) mkpart data ext4 0% 100%
```

Note: The filesystem type you pass to `mkpart` (here `ext4`) only tags the partition; it does not actually create a filesystem. You still need to run `mkfs` afterward, as shown in the next section.

Verify the result with the `print` command, which displays the disk's partition table:

Copy

```
$ (parted) print
```

The output confirms the new partition and its size:

Copy

Output

```
Model: Virtio Block Device (virtblk)
Disk /dev/sdb: 20 GiB
Partition Table: gpt
Disk Flags:
```

Number	Start	End	Size	File system	Name	Flags
1	1049kB	20 GiB	20 GiB		data	

Type `quit` to exit `parted`. Because `parted` writes changes immediately, there is no separate write step as there is in `fdisk`. To do the same thing in a single non-interactive command, which is ideal for automation, pass the commands with the `--script` flag:

Copy

```
$ sudo parted --script /dev/sdb mklabel gpt mkpart data ext4 0% 100%
```

How to format a partition



Creating a partition reserves the space, but you cannot store files on it until you format it with a filesystem. Formatting is done with the appropriate `mkfs` command, and the two most common choices are `ext4` and `xfs`.

Warning: Running `mkfs` erases everything on the target partition. Double-check the partition path (for example `/dev/sdb1`, not the whole disk `/dev/sdb`) before you run it.

To format the new partition with the `ext4` filesystem, run `mkfs.ext4` against the partition path:

```
$ sudo mkfs.ext4 /dev/sdb1
```

The command reports the filesystem details as it writes them:

Output

```
mke2fs 1.47.0 (5-Feb-2023)
Creating filesystem with 5242880 4k blocks and 1310720 inodes
Filesystem UUID: 9c5a3f7e-1b2c-4d8e-9f10-2a3b4c5d6e7f
Superblock backups stored on blocks:
    32768, 98304, 163840, 229376, 294912

Allocating group tables: done
Writing inode tables: done
Creating journal (16384 blocks): done
Writing superblocks and filesystem accounting information: done
```

If you prefer xfs, which is the default on Red Hat Enterprise Linux and Rocky Linux, format the partition with `mkfs.xfs` instead:

```
$ sudo mkfs.xfs /dev/sdb1
```

When to choose ext4 vs xfs

Both filesystems are mature and reliable, so the right choice depends on your workload. The ext4 filesystem is the default on Ubuntu and Debian and is a sensible general-purpose choice, especially for root and home partitions, because it is resilient and is the only one of the two that can be shrunk to reclaim space later. The xfs filesystem is the default on Red Hat based distributions and excels in high-throughput, high-I/O server environments and with very large filesystems; it can be grown while mounted but cannot be shrunk. As a rule of thumb, choose ext4 for general use or where you might need to shrink the filesystem, and choose xfs for large-scale, high-parallelism storage.

How to mount a partition in Linux



A formatted partition still needs to be mounted to a directory before you can use it. Mounting attaches the partition's filesystem to a location in your directory tree, called a mount point.

First, create a directory to serve as the mount point:

Copy

```
$ sudo mkdir -p /mnt/data
```

Then mount the partition to that directory:

Copy

```
$ sudo mount /dev/sdb1 /mnt/data
```

Confirm the partition is mounted and see how much space it has with `df -h`, which reports filesystem usage in a human-readable format:

Copy

```
$ df -h /mnt/data
```

The output shows the device, its size, and where it is mounted:

Copy

Output

Filesystem	Size	Used	Avail	Use%	Mounted on
/dev/sdb1	20G	24K	19G	1%	/mnt/data

This mount is temporary: it will not survive a reboot. To make it permanent, add an entry to `/etc/fstab` as described next.

How to mount a partition permanently using `fstab`

To mount a partition automatically every time the system boots, you add it to the `/etc/fstab` file. The recommended way to reference the partition is by its UUID, a unique identifier assigned when the partition is formatted, because UUIDs do not change when disks are reordered the way device names like `/dev/sdb1` can.

First, find the UUID of the partition with `blkid` :

Copy

```
$ sudo blkid /dev/sdb1
```

The output includes the `UUID` value you need:

Copy

Output

```
/dev/sdb1: UUID="9c5a3f7e-1b2c-4d8e-9f10-2a3b4c5d6e7f" TYPE="ext4" PARTLABEL=
```

Next, open `/etc/fstab` in a text editor:

Copy

```
$ sudo nano /etc/fstab
```

Add a line at the end of the file using the UUID from the previous step. The six fields are the source, the mount point, the filesystem type, the mount options, the dump flag, and the filesystem check order:

```
/etc/fstab
```

Copy

```
UUID=9c5a3f7e-1b2c-4d8e-9f10-2a3b4c5d6e7f /mnt/data ext4 defaults 0 2
```

Save and close the file. Rather than rebooting to test the entry, mount everything listed in `/etc/fstab` with `mount -a`, which catches errors safely:

Copy

```
$ sudo mount -a
```

If the command returns no output, the entry is valid and the partition will mount automatically on every boot. If it reports an error, fix the `/etc/fstab` line before rebooting, because an invalid entry can interrupt the boot process.

How to delete a partition in Linux



You can remove a partition with `fdisk` when you no longer need it or want to reorganize a disk. Deleting a partition removes it from the partition table and makes its data inaccessible.

Warning: Deleting a partition destroys the data it contains. Unmount the partition first and make sure you have selected the correct one before writing the change.

If the partition is mounted, unmount it before you delete it:

Copy

```
$ sudo umount /dev/sdb1
```

Open the disk in `fdisk`, then use the `d` command to delete a partition. If the disk has more than one partition, `fdisk` asks which partition number to remove:

Copy

Output

```
Command (m for help): d
Partition number (1-3, default 3): 1
Partition 1 has been deleted.
```

As with creating a partition, the deletion lives only in memory until you press **w** to write the change to disk. Press **q** instead if you change your mind and want to leave the partition in place.

Troubleshooting common partitioning errors

Partitioning operations occasionally fail with errors that have clear causes. Let's look at some of the most common ones, and how to resolve them.

“Device or resource busy” when writing the partition table

This error appears when you try to modify a disk that has a partition currently mounted or in use, such as an active swap partition. Unmount every partition on the disk with `sudo umount /dev/sdbN` and disable any swap on it with `sudo swapoff /dev/sdbN`, then run your `fdisk` or `parted` command again. You cannot repartition the disk that holds the running system without booting from a live environment.

Partition not visible after creation

Sometimes a newly created partition does not appear when you run `lsblk`, because the kernel has not yet reread the partition table. Force the kernel to reload it with the following command, replacing `/dev/sdb` with your disk:

Copy

```
$ sudo partprobe /dev/sdb
```

After running `partprobe`, run `lsblk` again and the new partition should appear. If it still does not, a reboot will always make the kernel reread the table.

“Partition table entries are not in disk order”

This message warns that the partition numbers do not run in the same order as their positions on the disk, which can happen after you delete and recreate partitions. It is a warning rather than a failure, and the partitions still work. In older versions of `fdisk` you could reorder entries with the `x` expert menu and the `f` command; on modern systems the warning is generally safe to ignore.

FAQs

1. What is the difference between `fdisk` and `parted` in Linux?

`fdisk` is an interactive command-line tool that supports both MBR and GPT partition tables and is available by default on most Linux distributions, which makes it ideal for quick, hands-on edits on a single disk. `parted` also supports MBR and GPT, can run non-interactively with the `--script` flag for automation, and is commonly preferred for scripted or repeatable workflows.

2. How do I list all disks and partitions in Linux before creating a partition?

Use `lsblk` to display all block devices in a tree format that shows each disk with its partitions, sizes, and mount points. For a more detailed view that includes the partition table type, run `sudo fdisk -l`, which lists every disk along with its partitions and types.

3. What partition table should I use: MBR or GPT?

Use GPT for disks larger than 2 TB or on systems that use UEFI firmware, because GPT removes the MBR size ceiling and supports up to 128 partitions. Use MBR for legacy BIOS systems or disks under 2 TB where compatibility with older systems is required, and when in doubt on modern hardware, GPT is the safer default.

4. How do I format a new partition as `ext4` in Linux?

After creating the partition, run `sudo mkfs.ext4 /dev/sdXN`, where `sdXN` is the partition identifier such as `/dev/sdb1`. Be careful to target the partition (for example `/dev/sdb1`) and not the whole disk (`/dev/sdb`), because `mkfs` erases the target.

5. How do I make a partition mount automatically on boot in Linux?

Get the partition UUID with `sudo blkid`, then add an entry to `/etc/fstab` using the format `UUID=<uuid> /mount/point ext4 defaults 0 2`. Run `sudo mount -a` to test the entry without rebooting, since using the UUID rather than a device name keeps the mount reliable even if disk ordering changes.

6. Can I create a partition on a disk that is already in use?

Creating a partition on a disk that has mounted partitions is possible but not recommended without first unmounting active partitions and disabling swap if applicable. For the disk that holds the running operating system, boot from a live USB or rescue environment first, because operating on it directly risks data corruption.

7. What is the difference between primary, extended, and logical partitions?

These distinctions apply only to MBR partition tables. MBR supports up to four primary partitions, and to go beyond that you convert one primary partition into an extended partition that holds logical partitions. GPT has no such limit and supports up to 128 partitions by default, so the distinction does not exist on GPT disks.

8. How do I verify a partition was created successfully in Linux?

Run `lsblk` to confirm the new partition appears in the block device tree, or run `sudo fdisk -l /dev/sdX` to inspect the partition table of a specific disk. After formatting and mounting, run `df -h` to confirm the partition is accessible and shows the expected size at its mount point.

Conclusion



You now know how to create a partition in Linux from start to finish using both `fdisk` and `parted`, including how to choose between an MBR and a GPT partition table, format the partition with `ext4` or `xfs`, mount it, make that mount persistent through `/etc/fstab`, delete a partition, and troubleshoot the most common errors. Because each partition behaves like a separate disk, this workflow lets you reserve space for specific tasks and keep data on one partition safe even if another becomes corrupted.

For repeatable, automated setups, consider scripting the process with `parted --script`, and for image-based or cloud workflows you may also want to explore

`systemd-repart` , a newer tool (introduced in [systemd 245](#)) that creates and grows GPT partitions declaratively from configuration files.

To learn more about Linux, we recommend checking out the following DigitalOcean tutorials:

- [How To Partition and Format Storage Devices in Linux](#)
- [50+ Essential Linux Commands You Should Know](#)
- [An Introduction to the Linux Terminal](#)
- [How To Create a New Sudo-Enabled User](#)
- [Initial Server Setup with Ubuntu](#)

Thanks for learning with the DigitalOcean Community. Check out our offerings for compute, storage, networking, and managed databases.

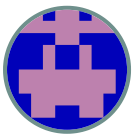
[Learn more about our products](#) →

About the author(s)



e09330e1749e4191a9dfe441e5129f Author

[See author profile](#)



Manikandan Kurup Editor
Senior Technical Content Engineer I

[See author profile](#)

With over 6 years of experience in tech publishing, Mani has edited and published more than 75 books covering a wide range of data science topics. Known for his strong attention to detail and technical knowledge, Mani specializes in creating clear, concise, and easy-to-understand content tailored for developers.

Category: Tutorial



Tags: UNIX/Linux



While we believe that this content benefits our community, we have not yet thoroughly reviewed it. If you have any suggestions for improvements, please let us know by clicking the "report an issue" button at the bottom of the tutorial.

Still looking for an answer?

Ask a question

Search for more help

Was this helpful?

Yes

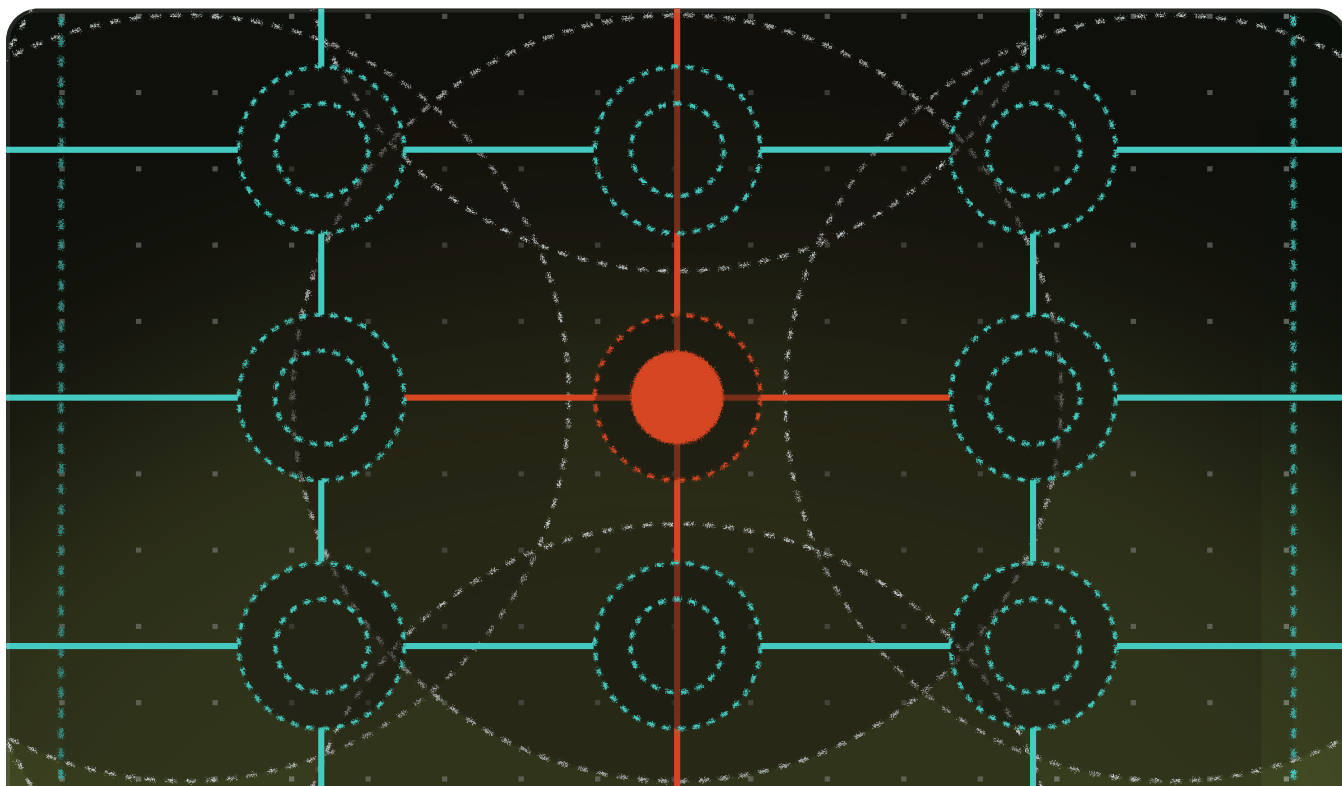
No

Comments(0)

Follow-up questions(0)



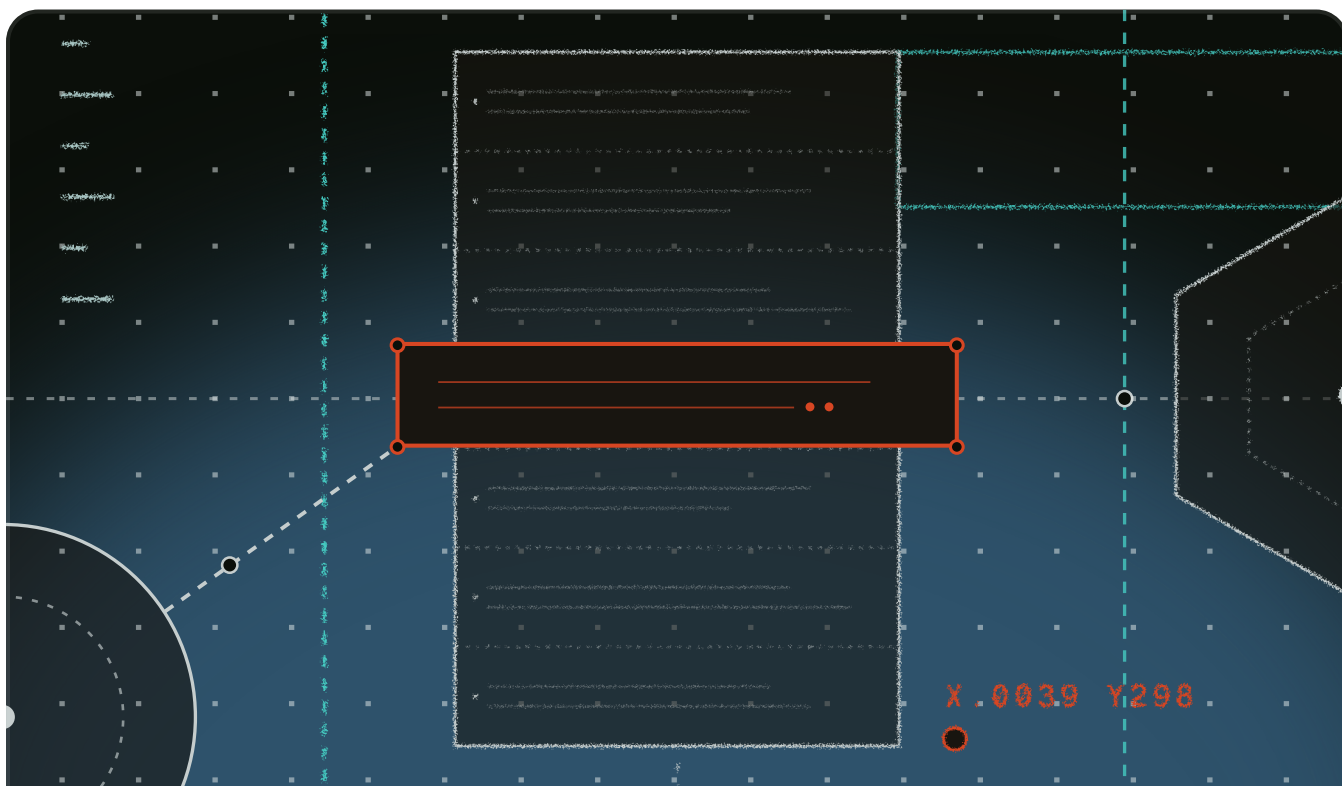
This work is licensed under a Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International License.



Станьте участником сообщества

Получайте деньги за написание технических руководств и выберите благотворительную организацию, связанную с технологиями, чтобы получить аналогичную сумму в качестве пожертвования.

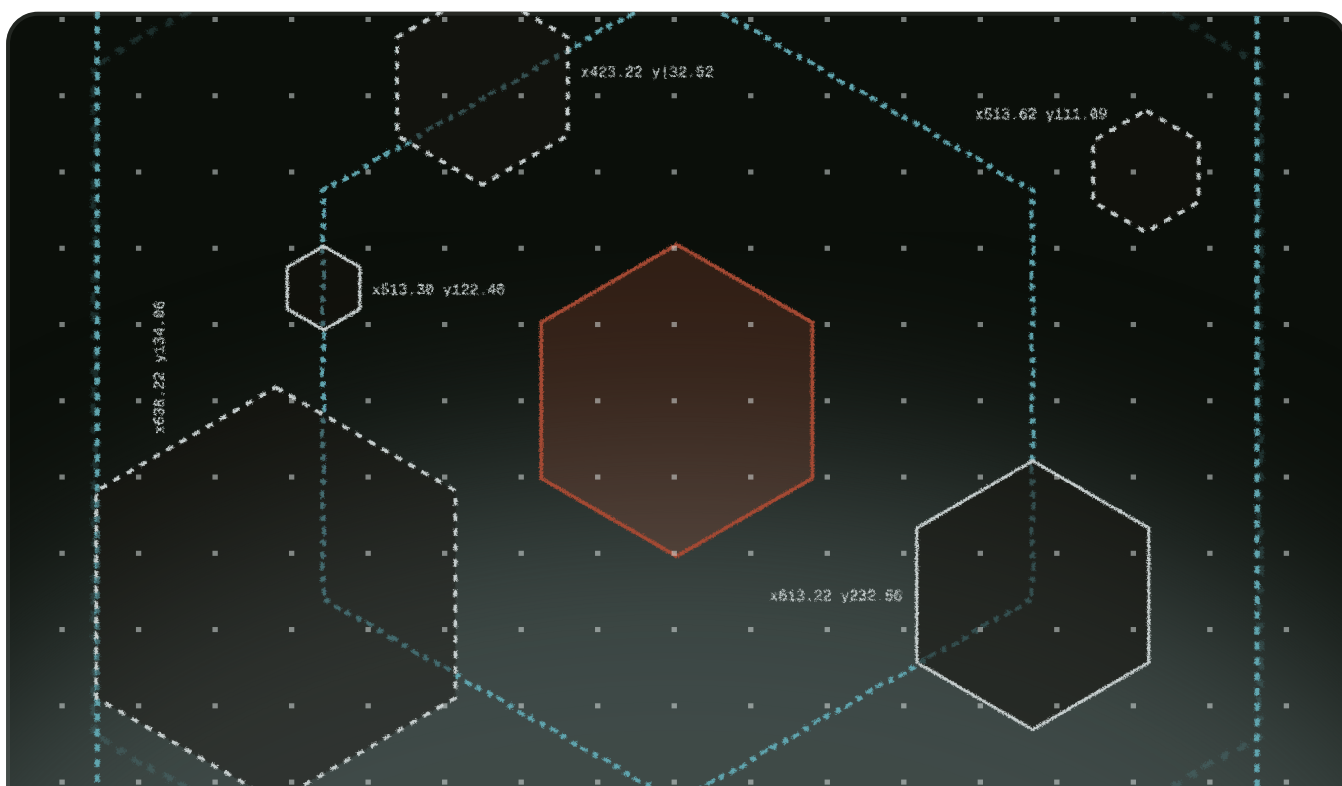
Регистрация →



Документация DigitalOcean

Полная документация по каждому продукту DigitalOcean.

Узнать больше →



Ресурсы для стартапов и компаний, использующих искусственный интеллект

The Wave has everything you need to know about building a business, from raising funding to marketing your product.

Learn more →

Stay up to date, sign up for our newsletter

Email*

Submit

The developer cloud

Scale up as you grow — whether you're running one virtual machine or ten thousand.

View all products

Start building today

From GPU-powered inference and Kubernetes to managed databases and storage, get everything you need to build, scale, and deploy intelligent applications.

Sign up

Company



Products



Resources



Solutions



Contact



© 2026 DigitalOcean, LLC. [Sitemap](#). [Cookie Preferences](#)

